

Project Handoff & Deployment Requirements

System: Jima Aba Jifar — Cafe & Restaurant Management System

Document: Handoff v1.1 · **Updated:** August 19, 2026

1. Repository Layout

```
workspace/ (pnpm monorepo)
├─ artifacts/
│   ├── habesha/           # Registered web artifact / routing shim
│   ├── nextjs-app/        # Real frontend – Next.js 15, React 19
│   │   ├── src/app/       # Pages: login, dashboard/* (orders, kitchen, reports, ...)
│   │   ├── src/components # Sidebar, manager-dashboard, shared UI
│   │   ├── src/lib/api.ts # All REST calls (Axios + JWT header)
│   │   ├── src/store/     # Zustand auth store
│   │   └─ public/         # Static assets (login-bg.jpg, ...)
│   └─ nestjs-server/      # Authoritative restaurant API – NestJS 10
│       └─ src/<module>/    # auth, users, orders, kitchen, payments,
                            # menus, tables, inventory, branches,
                            # restaurants, notifications, seed
├─ api-server/             # Separate minimal artifact; not the restaurant API
├─ docs/                   # This documentation set + screenshots
└─ package.json            # Workspace root
```

2. How to Run (Development)

```
# prerequisites: Node.js 18+, pnpm 8+, PostgreSQL 14+
pnpm install

# backend (port 3001, API prefix /api, Swagger at /api/docs)
DATABASE_URL=postgres://user:pass@host:5432/db \
JWT_SECRET=change-me \
pnpm --filter @workspace/nestjs-server run dev
```

```
# frontend (port 3000; proxies /backend/api/* to the backend)
BACKEND_URL=http://localhost:3001 \
pnpm --filter @workspace/nextjs-app run dev
```

Development seed operations require administrative authentication. Do not publish or reuse development credentials. Production provisioning must be controlled, auditable, and separate from normal application startup.

3. Environment Variables

Variable	Used by	Purpose
<code>DATABASE_URL</code>	Backend	PostgreSQL connection string. Append <code>?sslmode=require</code> for managed databases (SSL is honored automatically).
<code>JWT_SECRET</code>	Backend	Signing key for access tokens. Must be supplied as a secret; the current fallback is a production blocker.
<code>PORT</code>	Both	Listen port (backend defaults to 3001).
<code>BACKEND_URL</code>	Frontend	Where the Next.js rewrite proxies API calls (e.g. <code>https://api.example.com</code>).
<code>CORS_ORIGIN</code>	Backend	Allowed production frontend origin after wildcard CORS is removed.

4. Deployment Requirements

Requirement	Detail
Node runtime	Node.js 18 or newer, pnpm available at build time.
Database	PostgreSQL 14+ (managed service recommended: Neon, Supabase, RDS, Cloud SQL). ~100 MB is plenty to start.
Processes	

	Two services: frontend (Next.js server or Vercel) and backend (NestJS, long-running Node process).
Networking	Frontend must reach the backend over HTTPS; browser only ever talks to the frontend origin (same-origin proxy).
TLS	Terminate HTTPS at the platform/load balancer.
Memory	512 MB per service is sufficient for small deployments.

5. Production Checklist

✓	Item
☐	Require a strong <code>JWT_SECRET</code> and remove the hard-coded fallback.
☐	Stop startup password resets/account reactivation; keep seed and account provisioning controlled and production-gated.
☐	Turn off TypeORM <code>synchronize</code> and adopt migrations (<code>typeorm migration:generate</code>) — auto-sync on a live database can drop/alter columns unexpectedly.
☐	Enable database backups (daily snapshot is enough initially).
☐	Restrict CORS on the backend to the frontend's production origin.
☐	Restrict or disable Swagger and seed operations in production.
☐	Resolve high security-scan findings or record a reviewed risk acceptance.
☐	Run automated role, branch, order, payment, inventory, report, notification, and export checks.
☐	Set up log retention / error alerting on both services.
☐	Create real staff accounts with strong passwords; deactivate demo users.

Data safety: the system currently relies on `synchronize:true` . Before the first production deploy, generate the schema once, then disable

synchronize. Never point a synchronized dev backend at the production database.

6. Operational Notes for the Next Team

- **Verification habit:** run `npx tsc --noEmit` inside `artifacts/nextjs-app` after frontend changes. Avoid running `next build` while the dev server is running.
- **Polling:** all live pages poll every 10 s; there is no WebSocket layer to maintain.
- **Exports:** Excel/PDF exports happen in the browser (xlsx, jsPDF) — no server-side rendering service is needed.
- **Role logic:** sidebar visibility lives in `src/components/sidebar.tsx`; per-page role behavior is in each page component; backend guards are authoritative.
- **Profitability math:** food cost is estimated from stock deduction/waste movements $\times$ unit cost; accuracy depends on the store team recording usage.
- **API reference:** Swagger UI is served by the backend at `/api/docs`.

7. Support Artifacts Included

File	Content
docs/01-FRS...html	Functional requirements by subsystem and role
docs/02-SRS...html	Full software requirements incl. non-functional
docs/03-SDD...html	Architecture, ER, and sequence diagrams; design decisions
docs/04-User-Manual.html	Step-by-step guide with screenshots
docs/06-Deploy-on-Vercel.html	Vercel-specific deployment walkthrough

docs/07-Architecture-and- Routing.md	Canonical artifact, routing, and service ownership
docs/08-Environment-and- Secrets.md	Variable ownership and secret-handling rules
docs/09-API-and- Authorization-Boundary.md	API paths, authentication, authorization, and error contracts
docs/10-Database-Safety- Runbook.md	Migration, backup, restore, seed, and rollback controls
docs/11-Test-and-Release- Runbook.md	Required build, test, security, smoke, and rollback gates
docs/12-Production-Readiness- Status.md	Current READY / NEEDS ATTENTION / BLOCKED evidence matrix